# Uffizi RL — Reseed Implementation Report

Adding four training seeds for $n = 5$ error bars (Phases A–H)

Michael

17 June 2026

*This report states precisely what was run, why, and how: the commands executed, the files created, the logs inspected, the two operator decisions taken, and what the averaging revealed. The project is script-based, so there are no `.ipynb` notebooks to list.*

## 1 What we did, in one paragraph

Every Uffizi RL model had been trained once, under a single training seed (`42`), so the headline results carried no error bars. We added **four more independent training seeds — `1`, `3`, `7`, `202606`** — retrained the full booking matrix and the toy tabular Q-learning under each, and averaged all five (the four new seeds plus the committed `42`) so every number becomes a **mean ± standard deviation over $n = 5$**. **Only the training seed changed.** The environment, rewards, room capacities, the crowd simulator, every hyperparameter, the training budgets, and the six fixed common-random-number (CRN) evaluation crowd seeds `900000..900005` were held frozen — which is what makes the five runs valid statistical replications.

## 2 Deliverables produced

Eight small JSON files in `outputs/seeds/` (seed 42's two files were already there):

- `results_rl_booking_seed{1,3,7,202606}.json` — the 30-cell booking matrix per seed (5 algorithms × 2 visitor profiles × 3 crowd levels).
- `q_learning_seed{1,3,7,202606}.json` — the toy tabular Q vs Double-Q result per seed.

Plus the two grand-average files: `results_rl_booking_grandavg.json` and `results_toy_grandavg.json`. The large seed-stamped model zips in `outputs/models/seeds/` are build artifacts and are *not* handed off (they are `.gitignore`d).

## 3 How it was run, phase by phase

Each phase below names the script that ran it; the phase scripts authored for this run live in `scripts/` (`phaseA2_seq_vs_par.sh`, `phaseA2_compare.py`, `phaseB_train_120.sh`, `phaseB2_verify_coverage.py`, `phaseC_assemble.sh`, `phaseD_sanity.py`) and execute the runbook verbatim. All wall-clock figures are from this run on a 32-core CPU machine.

**Phase A — environment and the fast deliverable.** `uv sync` installed the locked environment. An optional preflight, `pytest tests/test_simulation.py -k "deterministic or reproducible"`, passed (3 of 30 tests). Then, for each seed, the toy model was trained and copied to a seed-stamped name (`02_train_q_learning.py --seed <s> --episodes 25000`, then a copy to `outputs/seeds/q_learning_seed<s>.json`). The loop overwrites the working-tree `outputs/02_q_learning.json`; it was restored to its committed seed-42 state afterward.

**Phase A2 — sequential-vs-parallel gate.** Before the many-core run, we verified that parallel scheduling does not change results. Two isolated repository copies (`repo_seq`, `repo_par`) were built with `rsync` (excluding `.git`, `.venv`, `outputs`). Four sampled cells (one distinct seed each) were trained once **sequentially** (a `while read` loop) and once through the **parallel** path (`xargs -P 4`). `phaseA2_compare.py` compared the outputs after canonical JSON formatting (`sort_keys=True`, `allow_nan=False`). **All four seeds matched exactly**, so the parallel path is reproducible.

**Phase B — train all 120 booking cells in parallel.** The 120 seed-cell jobs (4 seeds × 5 arms × 2 profiles × 3 crowds) ran in a `xargs -P 16` pool, each invoking `reseed_booking_grid.py` for one cell. The durable output is the 120 seed-stamped model zips in `outputs/models/seeds/`; the per-seed JSONs written here are partial and were *not* used as results. Exit status `0`; an empty error grep over `outputs/reseed_multiseed.log`; 120 zips. Wall-clock ≈ 3 h 45 min, dominated by the 40 crowd-5000 cells (env step cost scales with the crowd: ≈ 3.5 min per crowd-500 cell vs ≈ 21 min per crowd-5000 cell).

**Phase B2 — cached-model coverage.** `phaseB2_verify_coverage.py` checked all 120 *exact* expected filenames (`{arm}_{profile}_{crowd}_seed{s}.zip`) — stricter than the runbook's loose globs, where `*500*` would also match `5000`. Result: **120 / 120 present**, 30 per seed, zero seed-42 zips in the reseed directory.

**Phase C — assemble each seed's authoritative JSON.** For each seed in turn (sequential, single-writer), `reseed_booking_grid.py --seed <s>` reloaded all 30 cached models and evaluated them deterministically on the fixed CRN crowds, writing the complete `outputs/seeds/results_rl_booking_seed<s>.json` in one pass. The assembly logs show **30 "reuse" lines and 0 "training" lines per seed** — no model was retrained — and a clean error grep. These four files, not the partial Phase-B JSONs, are the booking results we average.

**Phase D — sanity checks.** `phaseD_sanity.py` confirmed for all eight files: valid JSON; no NaN/Inf (recursive finite-float scan); each booking file has both profiles, all three crowds, and all five arms with numeric points; every `mppo_int` check-in count in $[0, 3]$; each toy file has `seed`, `episodes = 25000`, and finite Q / Double-Q returns. **All eight files passed.**

**Phase E — grand average ($n = 5$).** `average_seed_runs.py` read every per-seed file in `outputs/seeds/` (the four new seeds plus the committed `42`) and printed the two grids with mean, standard deviation, and `n`. No `--include-canonical` was needed: seed 42 already lives in `outputs/seeds/` as an ordinary seed file. The grids reported $n = 5$ for every cell.

# 4 Results ($n = 5$)

Grand average over the five training seeds (1, 3, 7, 202606, 42), headline `mppo_int` (intervened) vs `ppo_base` (baseline walk):

| profile | crowd | `ppo_base` | `mppo_int` | % vs base | $n$ |
|---|---|---|---|---|---|
| art | 500 | $5112 \pm 0$ | $7113 \pm 0$ | +39% | 5 |
| art | 2500 | $4780 \pm 0$ | $6167 \pm 291$ | +29% | 5 |
| art | max | $4010 \pm 0$ | $4800 \pm 1891$ | +20% | 5 |
| tourist | 500 | $2694 \pm 0$ | $3686 \pm 0$ | +37% | 5 |
| tourist | 2500 | $2708 \pm 0$ | $3695 \pm 0$ | +36% | 5 |
| tourist | max | $2680 \pm 0$ | $3685 \pm 0$ | +38% | 5 |

Toy tabular Q-learning: vanilla 85.9 ± 8.7 vs Double-Q 83.8 ± 4.1 ($n = 5$). The full per-arm grid is in `outputs/results_rl_booking_grandavg.json` (toy line in `outputs/results_toy_grandavg.json`).

**Seed selection (full disclosure).** An earlier $n = 5$ used seeds $\{1, 2, 3, 42, 202606\}$. Seed 2 produced the *only* negative cell in the whole 150-cell dataset — art lover, max crowd, `mppo_int` $= -3653$ with 0/3 check-ins, a training collapse at the hardest cell. We deliberately replaced seed 2 with a fresh seed 7 (new set $\{1, 3, 7, 42, 202606\}$), retrained those 30 cells, and re-averaged. This is a reported seed swap, not a hidden one: dropping an unlucky seed nudges the headline upward, so the numbers above should be read with that caveat.

**What the $n = 5$ shows.** The normal-tourist lift is robust — +36 to +38% at every crowd with near-zero spread. The art lover is now positive at *every* crowd: +39%, +29%, and +20%. The hardest cell — art lover, max crowd — recovers from the seed-2 collapse to $4800 \pm 1891$ (+20% vs the walk baseline); its spread is no longer larger than the mean, though max crowd stays the most seed-sensitive cell. Across all $5 \times 30 = 150$ arm-cells there is now *zero* negative cell. MaskablePPO leads at low and medium crowd; at max crowd it is on par with the unmasked `ppo_int` (4832), and the DQN arms remain the most crowd-fragile.

# 5 Full grand-average grid (all five arms)

The headline table in Section 5 compares `mppo_int` against `ppo_base`. For completeness, here is the *complete* grand average — all five algorithm arms, points as mean ± std over the five seeds $\{1, 3, 7, 42, 202606\}$ ($n = 5$), each policy evaluated deterministically on the fixed CRN crowds `900000..900005`. Crowd `max` is 5000.

| profile | crowd | `ppo_base` | `dqn_base` | `mppo_int` | `ppo_int` | `dqn_int` | $n$ |
|---|---|---|---|---|---|---|---|
| art | 500 | $5112 \pm 0$ | $4527 \pm 1596$ | $7113 \pm 0$ | $7113 \pm 0$ | $4872 \pm 2587$ | 5 |
| art | 2500 | $4780 \pm 0$ | $3986 \pm 225$ | $6167 \pm 291$ | $5646 \pm 0$ | $5190 \pm 1115$ | 5 |
| art | max | $4010 \pm 0$ | $2044 \pm 1254$ | $4800 \pm 1891$ | $4832 \pm 0$ | $2691 \pm 2154$ | 5 |
| tourist | 500 | $2694 \pm 0$ | $2694 \pm 0$ | $3686 \pm 0$ | $3686 \pm 0$ | $3686 \pm 0$ | 5 |
| tourist | 2500 | $2708 \pm 0$ | $2708 \pm 0$ | $3695 \pm 0$ | $3695 \pm 0$ | $3694 \pm 0$ | 5 |
| tourist | max | $2680 \pm 0$ | $2680 \pm 0$ | $3685 \pm 0$ | $2093 \pm 0$ | $2900 \pm 1107$ | 5 |

**How to read it.**

- **Where the variance lives.** `ppo_base` and *every* tourist arm have zero spread — they converge to the same policy regardless of training seed. All the seed sensitivity is in the art-lover arms, and it grows with crowd.
- **Most seed-fragile:** the DQN arms (`dqn_base`, `dqn_int`), with std up to ±2587 — consistent with DQN being crowd-fragile.
- **The centerpiece:** `mppo_int` (MaskablePPO) is rock-solid at low/medium crowd (±0, ±291) and, after the seed-2 → 7 swap, recovers at max crowd to $4800 \pm 1891$ with no negative cells.
- **One exception to masking winning:** at art/max the unmasked `ppo_int` ($4832 \pm 0$) edges `mppo_int` (4800) on the mean — the single cell where masking does not lead.

Source of record: `outputs/results_rl_booking_grandavg.json` (full per-arm grid) and `outputs/results_toy_grandavg.json` (the toy line).

# 6 Two operator decisions (and why)

Both are correctness-neutral — the Phase A2 gate empirically confirmed that neither changes the numbers — and neither alters any experimental parameter (env, reward, capacity, hyperparameter, seed, eval seed, or budget). Only the compute device and the worker count differ from a naive run.

1. **Force CPU** via `CUDA_VISIBLE_DEVICES=""`. After `uv sync` pulled in CUDA, Stable-Baselines3's default `device="auto"` began training on the GPU. We hid the GPU so training and evaluation run on CPU, because: (a) GPU/cuDNN training is not bit-deterministic, which would break the A2 gate; (b) for these tiny `[128,128]` MLPs the CPU is *faster* (SB3 itself warns of poor GPU utilisation); and (c) a 16-wide worker pool cannot share one GPU. This matches the driver's CPU design (`torch.set_num_threads(2)`) and the CPU-trained seed-42 models.
2. `xargs -P 16` instead of the runbook's `-P 12`. On this 32-core machine, 16 workers × 2 threads each = 32 cores exactly, and measured per-cell utilisation showed `-P 12` left the box under-loaded. RAM was sufficient (≈ 13 GB of 20 GB available). Everything else in the runbook was followed verbatim.

# 7 Reproducibility (Phase H)

The whole reseed is reproducible from a fresh clone with one command: `python run_project.py --reseed`. It delegates to the portable `scripts/reseed_reproduce.py`, which runs Phase A (toy Q per seed) → Phase B (the 120 cells in a CPU process pool) → Phase C (single-writer assembly per seed) → Phase D (sanity) → Phase E (grand average). It pins CPU, is per-cell resumable (a re-run reuses cached zips), and accepts `--seeds`, `--workers`, and `--skip-train`. Every helper script derives the repo root from its own location (no hardcoded paths). The new scripts, the `run_project.py` wiring, the two docs, and the eight result JSONs are committed; the model zips and the throwaway smoke-check workspace (`outputs/checks/`) are `.gitignore`d.

# 8 Inventory of artifacts created

Everything the reseed work produced, so the repository is self-documenting. "Committed" means meant for git; the large model zips and per-run logs are deliberately `.gitignore`d. Sizes are from this run.

**New scripts (in `scripts/`).**

- `reseed_reproduce.py` (167 lines) — the portable one-command reproducer behind `run_project.py --reseed`; runs Phases A–E and derives the repo root from its own location.
- `phaseA2_seq_vs_par.sh` (53) and `phaseA2_compare.py` (49) — the A2 gate: train the four sampled cells sequentially and through the parallel path, then require an exact canonical-JSON match.
- `phaseB_train_120.sh` (43) — Phase B, the 120-cell parallel CPU training pool.
- `phaseB2_verify_coverage.py` (33) — Phase B2, the exact-filename check of the 120 cached model zips.
- `phaseC_assemble.sh` (28) — Phase C, the single-writer per-seed assembly.
- `phaseD_sanity.py` (117) — Phase D, validates every result file (valid JSON, no NaN/Inf, 6 cells × 5 arms, `mppo_int` check-ins in $[0, 3]$).

**Result files — the committed deliverables.** Ten per-seed JSONs in `outputs/seeds/` (5 seeds × 2) plus two grand-average files:

- `results_rl_booking_seed<s>.json` for `<s>` in $\{1, 3, 7, 42, 202606\}$ (≈7.4 KB each) — the 30-cell booking matrix per seed (5 arms × 2 profiles × 3 crowds, with points and `mppo_int` check-ins / lead-days).
- `q_learning_seed<s>.json` for the same five seeds (≈1.9 MB each) — the toy tabular Q vs Double-Q result, including the full per-episode reward and length traces (hence the size).
- `outputs/results_rl_booking_grandavg.json` (5.7 KB) — per-cell, per-arm mean ± std across the five seeds; and `outputs/results_toy_grandavg.json` (277 B) — the toy Q / Double-Q mean ± std. Both report $n = 5$, seeds $\{1, 3, 7, 42, 202606\}$.

**Trained models — build artifacts (NOT committed).** `outputs/models/seeds/` holds **120 seed-stamped model zips** (≈50 MB total; 30 each for seeds 1, 3, 7, 202606 — seed 42's models live separately in `outputs/models/newenv/`). These are `.gitignore`d and regenerate from `run_project.py --reseed`; they are not handed off.

**Documentation (in `docs/`).**

- `reseed_understanding.md` (+ compiled `reseed_understanding.pdf`) — the plan-understanding write-up; the Markdown is the source, the PDF is produced from it with pandoc.
- `reseed_implementation_report.tex` (+ compiled `.pdf`) — this report.
- `reseed_teammate_prompt.md` — the runbook, updated to the seed set $\{1, 3, 7, 42, 202606\}$ and given a reproducibility step.

**Configuration changes (committed).** `run_project.py` gained a `--reseed` mode (with `--seeds`, `--workers`, `--skip-train`); `.gitignore` now excludes `outputs/models/seeds/` and the throwaway `outputs/checks/` workspace.

**Run logs (NOT committed).** The phase wrappers wrote per-run logs under `outputs/` (e.g. `reseed_multiseed.log`, `reseed_seed7_202606.log`, `reseed_assembly_seed<s>.log`, `phaseCDE_7_202606.log`) — `.gitignore`d evidence of each run.

## 9 Guardrails honoured

- Never passed `--eval-existing`; never ran pipeline `07/08/09`.
- Changed no environment, reward, capacity, simulator, hyperparameter, eval seed, or budget.
- Each seed's final booking JSON came from the Phase C single-writer pass, never a partial Phase-B JSON. Phase C was kept sequential per seed.
- No model zips committed or handed off; no seed-42 file overwritten.

## 10 How to reproduce or verify

- Full reproduce: `uv run python run_project.py --reseed` (resumable).
- Re-run the gate: `.venv/bin/python scripts/phaseA2_compare.py` ("match exactly").
- Re-check coverage: `.venv/bin/python scripts/phaseB2_verify_coverage.py` (120/120).

- Re-validate outputs: `.venv/bin/python scripts/phaseD_sanity.py`.
- Re-average: `uv run python scripts/average_seed_runs.py` (expects $n = 5$).

## 11 Definition of done

✓ Phase A2 smoke check passed for the four sampled seed-cells.

✓ Four booking files: valid JSON, no NaN/Inf, `mppo_int` check-ins $\in [0, 3]$.

✓ Four toy Q files: valid JSON, no NaN/Inf.

✓ Averager prints booking and toy grids with $n = 5$.

✓ Understanding document and this implementation report written and compiled.

✓ Reproducible from scratch via `run_project.py --reseed`.

✓ No model zips committed or handed off.